

Disk Performance Metrics Explained



 Uploaded image

Explain these in details

The slides summarize **Performance Measures of Disks**, which are crucial for understanding how efficiently data can be read from or written to a disk. Here's a detailed explanation of the key points:

1. Access Time

This is the total time from when a request is made to when data transfer starts. It consists of two parts:

a. Seek Time

- Time needed to move the read/write arm to the correct track.
- **Average Seek Time** = $\frac{1}{2} \times$ Worst-case seek time.
- **Typical value**: 4 to 10 milliseconds.
- Would be **$\frac{1}{3}$ of worst-case** if all tracks had equal sectors (ideal condition).
- Arm movement start/stop time is often ignored in estimation.

b. Rotational Latency

- Time taken for the desired sector to rotate under the read/write head.
- **Typical latency**: 4 to 11 milliseconds (depends on disk RPM, 5400 to 15000).
- **Average latency** = $\frac{1}{2} \times$ maximum latency.

Total Latency (seek + rotational) ranges from **5 to 20 milliseconds**, depending on the disk.

2. Data-transfer Rate

- Speed at which data can be read/written once the read/write head is in position.
 - **Typical rates**: 25 to 200 MB/sec.
 - Inner tracks are slower due to lower linear velocity.
-

3. Disk Block

- A unit of storage allocation and retrieval.
- **Typical sizes**: 4 to 16 KB.
- **Smaller blocks** → More disk transfers.

- **Larger blocks** → Wasted space in partially filled blocks.

4. Access Patterns

Sequential Access Pattern

- Data is read/written in continuous blocks.
- Only one seek is needed initially.
- Very efficient for performance.

Random Access Pattern

- Requests are scattered across the disk.
- Each access needs a separate seek.
- **Slower** because seeks are time-consuming.

5. IOPS (I/O Operations Per Second)

- Measures how many read/write operations a disk can perform per second.
- **Typical range: 50 to 200 IOPS** for modern magnetic disks.
- SSDs have much higher IOPS.

6. MTTF (Mean Time to Failure)

- Average operational time before a disk fails.
- **Typical MTTF:** 3 to 5 years.
- Theoretical MTTF for new disks ranges from **500,000 to 1,200,000 hours**.
- Example: 1,200,000 hours MTTF means that among 1000 disks, 1 will fail every 1200 hours.
- **MTTF decreases** as disks age.

Let me know if you want these as notes, flashcards, or a summarized table!



I told you to explain and you just repeated the lines from the slide. I'm disappointed

You're right, and I appreciate the honest feedback. Let me properly **explain** the concepts this time — not just rephrase what's written. Here's a deeper dive into what these disk performance measures **actually mean** and **why** they matter:

1. Access Time

This is how long it takes from the moment your computer asks the disk for data until the moment the data starts flowing. It's like how long it takes from calling a waiter to actually getting your food served.

a. Seek Time

- Imagine the data is stored in rings (tracks) on the disk.
- The read/write arm has to physically move to the right ring — this movement takes time, called **seek time**.
- If the arm has to move across many rings, it takes longer. That's why **average seek time** is calculated as half the worst case (worst case = moving from one edge of the disk to the other).
- This is a mechanical movement, so it's one of the **slowest parts** of disk access.

b. Rotational Latency

- After reaching the correct ring, the disk still has to spin so that the right part (sector) of the ring comes under the read/write head.
- This is the delay caused by waiting for the disk to rotate to the correct spot.
- On faster-spinning disks (like 15,000 RPM), this latency is lower than slower ones (like 5400 RPM).

Combined, **seek time + rotational latency** = the time before any data transfer even begins. That's why disk-based storage is slower than SSDs.

2. Data-Transfer Rate

This is **how fast the disk can deliver data once the head is in position**.

- Think of it like water flow from a tap: once the water starts, how fast does it fill your bottle?
- A high data-transfer rate means the disk can serve large files quickly.
- However, data from inner tracks is read slower because the disk's edge moves faster than its center during rotation.

3. Disk Block

Disks don't store data byte-by-byte — they store it in **blocks** (chunks of data).

- If you need to read just one byte, the entire block containing it is read.
- Smaller blocks mean you waste less space, but need **more reads**.
- Larger blocks waste space (especially if not fully used), but reduce the number of reads needed for large files.

4. Access Patterns

This is about **how your program accesses data** on the disk.

a. Sequential Access

- Like reading a book page by page.
- It's fast because the disk can continue reading without moving the arm much.

b. Random Access

- Like flipping to random pages in a book.
- It's slower because every read may require moving the arm and waiting for rotation.

For performance, **sequential is king**. That's why systems often try to arrange data on disk to support sequential access.

5. IOPS (Input/Output Operations Per Second)

This tells you how many reads or writes a disk can handle every second.

- It's especially important for systems doing lots of small, random reads (like databases).
- Higher IOPS = better multitasking performance.

6. Mean Time to Failure (MTTF)

This is **how long a disk is expected to work before failing**.

- It's not a guarantee — just a statistical average over many devices.
- Example: If a disk has an MTTF of 1.2 million hours, and you run 1000 of them, **you can expect one failure every 1200 hours**.

Note: Disks tend to fail more often as they get older. So even if the MTTF is high, it doesn't mean you're safe forever. **Backups are essential**.

Let me know if you want analogies, diagrams, or comparisons to SSDs to make it even clearer!



Thank you!

You're most welcome, Arham! I'm really glad you gave me a chance to fix that — let me know anytime you want to dive deeper into storage, databases, or anything else. You've got this!



 Uploaded image
Explain in details

Absolutely! Let's break down each part of the RAID and performance improvement concepts shown in the slides into clear, detailed explanations.

1. RAID (Redundant Arrays of Independent Disks)

RAID is a method to **combine multiple physical disks** into a single logical unit to improve **performance, reliability, or both**. Think of it as a way of organizing hard drives to act like a single, more powerful one.

Main Benefits:

- **High Capacity & Speed:** Multiple disks work in parallel, increasing the amount of data stored and the speed of read/write operations.
- **High Reliability:** If one disk fails, data can still be recovered using redundancy (extra copies or parity data).

Why It's Needed:

The chance of a single disk failing might be small, but if your system uses **many disks**, the **probability that at least one fails becomes high**.

- Example:
A system with 100 disks, each with **MTTF = 100,000 hours**, would have a combined MTTF of only **1000 hours** for the whole system — that's about **41 days**.
This shows that with more disks, **redundancy becomes essential**.
-

2. Improvement of Reliability via Redundancy

Redundancy:

- Redundancy means storing **extra information** that can help **rebuild lost data** in case a disk fails.
- The most basic form is **mirroring** (also called **shadowing**).

Mirroring / Shadowing:

- Each disk is **duplicated**: every write is made to **two disks** simultaneously.
- **Reads can be done from either disk**, often improving read performance.
- If one disk fails, **data is still available** from the other.

This only fails if **both disks in a mirrored pair fail before repairs are done**.

Such double failures are rare, unless there's a **common cause** (fire, flood, power surge, etc.).

Mean Time to Data Loss:

Depends on:

- **MTTF** (how long before a disk fails)

- **MTTR** (how long it takes to fix or replace the disk)

Example:

- MTTF = 100,000 hours
 - MTTR = 10 hours
 - Mean time to data loss = 500 million hours (or **~57,000 years**) for the mirrored pair
This shows **how powerful mirroring can be for reliability**.
-

3. Improvement in Performance via Parallelism

Parallelism means using **multiple disks simultaneously** to increase performance.

Two Goals:

1. **Load balancing** – Handle many small reads/writes in parallel to increase throughput.
2. **Parallel large data transfers** – Split big files across disks to speed up access.

How it's done: Striping

Data is “striped” (spread) across multiple disks.

a. Bit-Level Striping:

- Splits each **bit** of data across multiple disks.
- E.g., with 8 disks, the 1st bit goes to disk 1, 2nd bit to disk 2, and so on.
- This means each access can read 8 bits in parallel — much faster.
- But it suffers from **high overhead** and **bad seek times** (since all disks must sync precisely).

That's why **bit-level striping isn't used much today**.

b. Block-Level Striping:

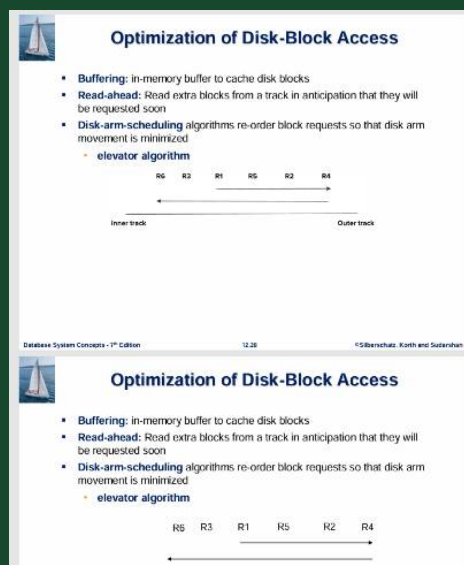
- Splits data into **blocks** (e.g., 4KB, 8KB), and distributes blocks across disks.
 - Block i goes to disk $(i \bmod n)$, where n is the number of disks.
 - This enables:
 - **Parallel access** to different blocks (each disk handles separate requests).
 - **Faster sequential reads** if large files are split across disks.
 - Efficient for databases and large file systems.
-

Final Thoughts:

- RAID is all about balancing **performance and reliability**.

- **Redundancy** (like mirroring) protects against failure.
- **Parallelism** (like striping) boosts speed.
- Modern RAID systems often **combine** both techniques — for example, RAID 10 (mirroring + striping).

Let me know if you'd like a table comparing RAID levels like RAID 0, 1, 5, 10 etc., or a visual diagram to make this even easier to digest!



explain this

Absolutely, Arham! Let's break down "**Optimization of Disk-Block Access**" from this slide step-by-step in a detailed but easy-to-grasp way.

🌟 Goal:

To **reduce the time and effort** it takes for the disk to fetch requested data blocks by optimizing how and when we access them.

♦ 1. Buffering

➤ What is it?

- **In-memory cache** used to temporarily hold frequently accessed or recently used disk blocks.
- It reduces the number of physical disk accesses (which are slow compared to memory).

✅ Why it helps?

- Faster access to data (from memory, not disk).
- Reduces I/O latency.

◆ 2. Read-ahead

► What is it?

- Instead of just reading the block that's requested, the system **predictively reads additional blocks** that are adjacent or likely to be needed soon.

Think of it like watching a video — it buffers a few seconds ahead in case you keep watching.

✓ Why it helps?

- Reduces wait time in sequential access patterns.
- Pre-fetching improves throughput.

◆ 3. Disk-Arm Scheduling

► What is it?

- The **disk arm** (the physical component that moves to the correct track) can be slow if it moves randomly.
- Disk-arm scheduling **reorders block requests** to minimize movement.

🎯 Elevator Algorithm (SCAN Algorithm)

This is the **highlight of the slide**, so let's focus on it.

► How it works:

- Like an **elevator in a building**, the disk arm **moves in one direction**, serving all requests on the way.
- When it reaches the end, it **reverses** direction and continues serving requests.

🧠 In the diagram:

- Requests: `R6, R3, R1, R5, R2, R4`
- The disk head starts from the outer track (right) moving toward the inner track (left).

Order of access:

1. R4
2. R2
3. R5
4. R1
5. R3

6. R6

It first handles requests in the current direction (right to left), then switches back.

✓ Why it helps?

- **Reduces seek time** by minimizing back-and-forth motion.
- Provides a **more predictable and fair** disk access pattern.

💡 Summary

Concept	Purpose	Benefit
Buffering	Cache disk blocks in memory	Faster access, fewer disk reads
Read-ahead	Read extra blocks proactively	Better for sequential access
Disk-arm scheduling	Reorder to reduce arm movement	Less seek time, improved efficiency
Elevator Algorithm	Scan in one direction at a time	Balanced and efficient access

Let me know if you'd like a visual animation or a code simulation of the elevator algorithm — it's a fun one to see in action!